AArch64 Memory & Cache System — Technical Reference Workbook 1
Chapter 4 — Instruction Visibility: PoU, L1I Behavior, FrontEnd Pipeline, and ISB Semantics (Based on ARM64 CortexA72) 1
4.1 Scope 1
4.2 Instruction Visibility Domains 1
4.2.1 L1I — CoreLocal Instruction Cache 1
4.2.2 PoU — Point of Unification (L2) 2
4.3 Instruction Fetch Path (CortexA72) 2
4.4 L1I Behavior 2
4.4.1 NonCoherent 2
4.4.2 Not Updated by DC Operations 2
4.4.3 Only Updated by Instruction Fetch 2
4.4.4 IC IVAU Behavior 3
4.5 PoU Semantics (Point of Unification) 3
4.6 FrontEnd Pipeline Behavior 3
4.7 ISB — Instruction Synchronization Barrier 4
4.7.1 Architectural Purpose 4
4.7.2 Required After IC IVAU 4
4.7.3 CortexA72 Behavior 4
4.8 Correct Instruction Modification Sequence 4
4.9 SMP Instruction Visibility 5
4.10 Microarchitectural Notes (CortexA72) 5
4.11 Exercises (Technical Reference) 6
Exercise 4.1 — PoU Role 6
Exercise 4.2 — IC IVAU Limitations 6
Exercise 4.3 — ISB Requirement 6
Exercise 4.4 — SMP Visibility 6
Exercise 4.5 — FrontEnd Pipeline 6

AArch64 Memory & Cache System — Technical Reference Workbook

Chapter 4 — Instruction Visibility: PoU, L1I Behavior, Front-End Pipeline, and ISB Semantics (Based on ARM64 Cortex-A72)

4.1 Scope

This chapter defines the architectural and microarchitectural mechanisms governing instruction visibility on ARM64 Cortex-A72. It covers:

- Instruction visibility domains
- L1I behavior
- PoU (Point of Unification) semantics
- Front-end pipeline behavior
- The role of IC IVAU
- The architectural requirement for ISB
- Correct sequencing for instruction modification

This chapter builds directly on the cache-maintenance semantics defined in Chapter 3.

4.2 Instruction Visibility Domains

Instruction visibility is governed by two architectural domains:

4.2.1 L1I — Core-Local Instruction Cache

- Private to each core
- Non-coherent
- Not snooped
- May contain stale instructions
- Does not automatically observe updates in L1D or L2

4.2.2 PoU — Point of Unification (L2)

- Shared by all cores in the cluster
- Unified domain for instruction and data streams
- Instruction refills originate from PoU
- Required for instruction visibility across cores

PoU is the **architectural source of truth** for instruction fetch.

4.3 Instruction Fetch Path (Cortex-A72)

Instruction fetch proceeds through the following microarchitectural stages:

1. **Branch predictor / BTB / return stack**
2. **Instruction prefetch unit**
3. **L1I lookup**
4. **L2 (PoU) lookup on miss**
5. **Front-end decode pipeline**
6. **Issue to execution pipeline**

Key property:

The front-end pipeline may contain prefetched instructions that are not yet architecturally visible.

This is why ISB is required after IC IVAU.

4.4 L1I Behavior

4.4.1 Non-Coherent

L1I is not coherent with L1D or L2. It does not receive snoops and does not observe data writes.

4.4.2 Not Updated by DC Operations

DC CVAU, DC CVAC, DC CIVAC do **not** update L1I.

4.4.3 Only Updated by Instruction Fetch

L1I is updated only when:

- A fetch misses in L1I
- The core refetches instructions from PoU
- ISB forces a pipeline flush

4.4.4 IC IVAU Behavior

IC IVAU invalidates the L1I tag entry for a given VA.

It does **not**:

- Flush the front-end pipeline
- Force immediate refetch
- Affect BTB or prediction state

ISB is required for architectural correctness.

4.5 PoU Semantics (Point of Unification)

PoU is the domain where:

- Instruction and data streams unify
- Instruction refetches originate
- DC CVAU pushes updated instruction bytes
- IC IVAU forces L1I to consult PoU
- ISB forces the front-end to refetch from PoU

PoU is always the **L2 cache** on Cortex-A72.

4.6 Front-End Pipeline Behavior

The front-end pipeline may contain:

- Prefetched instructions
- Speculatively fetched instructions
- Instructions fetched before IC IVAU

- Instructions fetched before DC CVAU writeback completes

These instructions are **not flushed** by IC IVAU.

They are flushed only by:

- **ISB**
- Certain exception boundaries
- Context switch mechanisms (implementation-specific)

4.7 ISB — Instruction Synchronization Barrier

4.7.1 Architectural Purpose

ISB ensures:

- All previous instruction fetches are discarded
- The front-end pipeline is flushed
- The next instruction fetch consults PoU
- Updated instructions become architecturally visible

4.7.2 Required After IC IVAU

ISB is mandatory after IC IVAU because:

- IC IVAU invalidates L1I tags
- But the pipeline may still contain stale instructions
- ISB forces a refetch from PoU

4.7.3 Cortex-A72 Behavior

ISB flushes:

- BTB predictions
- Return stack entries
- Prefetch buffers
- Decode pipeline
- Issue queue entries related to instruction fetch

4.8 Correct Instruction Modification Sequence

The architecturally required sequence is:

```
DC   CVAU, xN     ; Push updated instruction bytes to PoU
IC   IVAU, xN     ; Invalidate stale L1I line
DSB  ISH          ; Ensure DC/IC operations complete
ISB               ; Flush pipeline and refetch from PoU
```

This sequence ensures:

- Updated instructions propagate from L1D → L2
- L1I drops stale instruction lines
- Front-end pipeline refetches updated instructions
- All cores observe correct instruction stream

4.9 SMP Instruction Visibility

When CPU0 modifies instructions intended for CPU1:

CPU0 must execute:

```
DC   CVAU, xN
IC   IVAU, xN
DSB  ISH
ISB
```

CPU1 must execute:

```
ISB
```

This ensures CPU1 refetches instructions from PoU.

4.10 Microarchitectural Notes (Cortex-A72)

Although hidden behind architectural barriers, Cortex-A72 implements:

- Non-inclusive L2
- No L1I snooping
- FEQ pipelining for DC operations
- Register slices for large L2 configurations
- Speculative front-end fetch
- BTB and return stack prediction pipelines

These do not alter architectural requirements.

4.11 Exercises (Technical Reference)

Exercise 4.1 — PoU Role

Explain why PoU is the only architecturally valid source for instruction refetch.

Exercise 4.2 — IC IVAU Limitations

Describe why IC IVAU alone cannot guarantee instruction visibility.

Exercise 4.3 — ISB Requirement

Explain the architectural reason ISB must follow IC IVAU.

Exercise 4.4 — SMP Visibility

Describe how CPU1 becomes aware of updated instructions modified by CPU0.

Exercise 4.5 — Front-End Pipeline

Explain why stale instructions may remain in the pipeline after IC IVAU.